

4.6.25

ABSTRACTION

Definition

- In general, it is the concept of hiding information implementation details and exposing functionality.

- In Java, it is the concept of hiding method definition and exposing method declaration.

NOTE

• In Java we can perform abstraction using

i) abstract methods

ii) abstract class

iii) Interface

• Abstract methods with abstract class will provide partial abstraction (It having concrete method from obj class)

• Abstract methods with interface will provide pure abstraction.

abstract methods.

- abstract methods are declaring with abstract keyword
- abstract methods will have only method declaration
- we can declare abstract methods using inside abstract class or interface.
- abstract methods cannot be
 - i) static
 - ii) private and
 - iii) final
- abstract methods are incomplete methods.

abstract class

① eg

```
abstract class Demo
{
```

```
    abstract void test();
```

```
    abstract void desp();
```

```
}
```

// abstract class contain only abstract method


```
class Sample extends Demo
```

```
{  
    @Override
```

```
    void test()
```

```
{
```

```
        S.o.pln("Test executing()...");
```

```
}
```

```
    @Override
```

```
    void disp()
```

```
{
```

```
        S.o.pln("disp executing...");
```

```
}
```

```
}
```

```
public class Mainclass1
```

```
{
```

```
    P.S.V.M.C()
```

```
{
```

```
        Demo obj = new Sample();
```

```
        obj.test();
```

```
        obj.disp();
```

```
}
```

```
}
```

```
② eg abstract class Alpha
```

```
{
```

```
    abstract void play();
```

```
    abstract
```

```
    void send()
```

```
{
```

```
        S.o.pln("executing send()...");
```

```
}
```

```
}
```

```
class Beta extends Alpha
```

```
{
```

```
    @Override
```

```
    void play()
```

```
{
```

```
        S.o.pln("executing play()...");
```

```
}
```

```
}
```

// abstract class contains

both abstract method and
concrete method.


```
public class Mainclass2
```

```
{
```

```
    P.s.v.m()
```

```
{
```

```
    Alpha obj = new Beta();
```

```
    obj.Play();
```

```
    obj.Send();
```

```
}
```

```
}
```

abstract class only contain not contain abstract method

③ eg. abstract class Delta

```
{
```

```
    void help()
```

```
{
```

```
        S.o.pln("executing help()...");
```

```
}
```

```
    void run()
```

```
{
```

```
        S.o.pln("executing run()...");
```

```
}
```

```
}
```

```
class Example extends Delta
```

```
{
```

```
}
```

```
public class Mainclass3 {
```

```
    P.s.v.m()
```

```
{
```

```
    Delta obj = new Example();
```

```
    obj.help();
```

```
    obj.run();
```

```
}
```

```
}
```

// abstract method

contain only

concrete method

① eg:-

abstract class Hotstar

{

void login()

{
=
=

}

abstract void watch();

}

class HotstarFree extends Hotstar

@Override

void watch()

{

S.o.Pl("Ads+Trailer");

}

class HotstarVip extends Hotstar

{

@Override

void watch()

{

S.o.Pl("Ad+Regional");

}

}

class HotstarPremium extends Hotstar

{

@Override

void watch()

{

S.o.Pl("AdFree+FullAccess");

}

}

class ContentManager

Static void accessControl(Hotstar h3)

{
if (h3 != null)

{

h3.login();

h3.watch();

}

}

public class MainClass

P.S.V.MC()

{

HotstarFree obj1 = new HotstarFree();

ContentManager.accessControl(obj1);

HotstarVip obj2 = new HotstarVip();

ContentManager.accessControl(obj2);

HotstarPremium obj3 = new HotstarPremium();

ContentManager.accessControl(obj3);

}

5) eg

```
abstract class Android
{
    void services()
    {
    }
    abstract void ui();
}
```

```
class vivo extends Android
{
    @Override
    void ui()
    {
        s.o.pln("Fun touch os");
    }
}
```

```
class oppo extends Android
{
    @Override
    void ui()
    {
        s.o.pln("color os");
    }
}
```

```
class samsung extends Android
{
    @Override
    void ui()
    {
        s.o.pln("one os");
    }
}
```

```
class Shop
{
    static void displayPhoneProperties(Android a)
    {
        if (a != null)
        {
            a.service();
            a.ui();
        }
    }
}
```

public class Mainclass

{

Vivo v = new Vivo();

shop.displayPhoneProperties(v);

OPPO o = new oppo();

shop.displayPhoneProperties(o);

Samsung s = new Samsung();

shop.displayPhoneProperties(s);

6 eq)

abstract class Testscript

```
{
    abstract void test();
    void report();
}
```

```
class LoginPageTest extends Testscript
{
    @Override
    void test()
    {
        s.o.pln("Testing login page");
    }
}
```

```
class HomePageTest extends Testscript
{
    @Override
    void test()
    {
        s.o.pln("Testing Home page");
    }
}
```

```
class ProfilePageTest extends Testscript
{
    @Override
    void test()
    {
        s.o.pln("Testing Profile page");
    }
}
```

class Testsuite

```
{
    static void executeTest(Testscript ts)
    {
        if (ts != null)
        {
            ts.test();
            ts.report();
        }
    }
}
```

public class MainClass

```
{
    P.S.V.MC()
    {
        LoginPageTest l = new LoginPageTest();
        Testsuite.executeTest(l);

        HomePageTest H = new HomePageTest();
        Test Suite . executeTest (H);

        ProfilePageTest P = new ProfilePageTest();
        Testsuite . executeTest (P);
    }
}
```


- abstract class is declared with abstract keyword.
- Inside the abstract class there is no restriction on the data members and member function.
- we cannot create an object for an abstract class directly.
- we can access the properties of an abstract class through inheritance and overriding.

Contract

- Whenever a concrete class inherits from an abstract class it is mandatory to override the inherited abstract methods in the concrete class.
- An abstract class is a partial abstract body and offers partial abstraction.

Why abstract class is considered as a partial abstract body?
Because of 2 reasons,

1. It will have a constructor.
2. It will contain concrete methods inherited from object class.
3. It allows both abstract method and concrete method.

Why constructor is needed in an abstract class?

Because of 2 Reasons

1. To initialize non-static data members.
2. To support constructor chaining.

• B.

can we declare an abstract class with zero abstract methods?

yes, but it is not common. however we can use this to

enforce inheritance.